

# An extended model-based characterization of fine-grained access control for SQL queries

Hoàng Nguyễn<sup>1</sup>   Manuel Clavel<sup>2</sup>

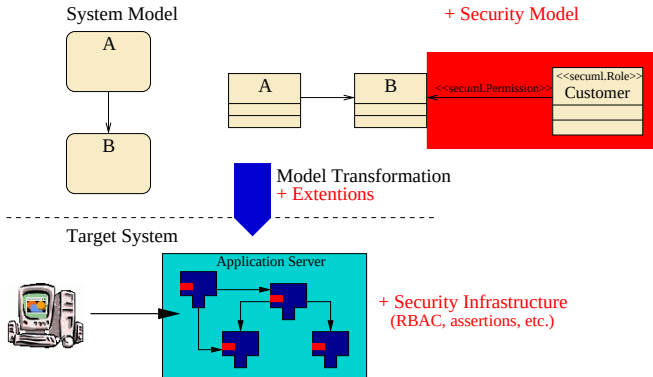
<sup>1</sup>ETH Zurich, Switzerland

<sup>2</sup>Eastern International University, Vietnam

04 October 2023, Västerås, Sweden

# Background

## Model-driven Security



# Background

## Model-driven Security

MDS delivers its potential for specific domains: secure access to data.

---

<sup>1</sup>Basin et. al. Is modeling access control worth it?. ACM CCS 2023.

# Background

## Model-driven Security

MDS delivers its potential for specific domains: secure access to data.

- Reduce software development time.

---

<sup>1</sup>Basin et. al. Is modeling access control worth it?. ACM CCS 2023.

# Background

## Model-driven Security

MDS delivers its potential for specific domains: secure access to data.

- Reduce software development time.
- Reduce software cost.

---

<sup>1</sup>Basin et. al. Is modeling access control worth it?. ACM CCS 2023.

# Background

## Model-driven Security

MDS delivers its potential for specific domains: secure access to data.

- Reduce software development time.
- Reduce software cost.
- Improve software security and quality.

---

<sup>1</sup>Basin et. al. Is modeling access control worth it?. ACM CCS 2023.

# Background

## Model-driven Security

MDS delivers its potential for specific domains: secure access to data.

- Reduce software development time.
- Reduce software cost.
- Improve software security and quality.

Recent empirical study shows that students perform better when task to modeling access control policy rather than implement them manually, in term of implementation correctness and development effort. <sup>1</sup>

---

<sup>1</sup>Basin et. al. Is modeling access control worth it?. ACM CCS 2023.

# Motivation

## SQL database schema

```
Student (Student_id, name, email)
Lecturer (Lecturer_id, name, email)
Enrolment (students, lecturers).
```

- There are students (Student) and lecturers (Lecturer).
  - ▶ Students and lecturers: each has a unique *id*.
  - ▶ Students and lecturers: each has a *name* and an *email*.
- (Enrolment) Lecturers can teach students and students can be taught by lecturers.
  - ▶ Lecturers and students are represented by their *ids*.

# Motivation

## SQL queries

```
Student (Student_id, name, email)
Lecturer (Lecturer_id, name, email)
Enrolment (students, lecturers).
```

Query#1: it returns the emails of the lecturers named *Juan* (if any).

```
SELECT email FROM Lecturer WHERE name = 'Juan'
```

# Motivation

## SQL queries

```
Student (Student_id, name, email)
Lecturer (Lecturer_id, name, email)
Enrolment (students, lecturers).
```

Query#2: it returns the email of the lecturer whose *id* is 1 if he/she happens to teach the student whose *id* is 1.

```
SELECT DISTINCT email FROM Lecturer
JOIN (SELECT * FROM Enrolment
      WHERE students = 1 and lectures = 1) AS TEMP
ON TEMP.lecturers = Lecturer_id
```

# Motivation

## SQL queries

Query#3: it returns the email of the lecturer whose *id* is 1, if there is at least one student that is taught both by this lecturer and the lecturer whose *id* is 2.

```
SELECT DISTINCT email FROM Lecturer
JOIN (SELECT TEMP1.lecturers AS lecturers
      FROM (SELECT * FROM Enrolment WHERE lecturers = 1) AS TEMP1
      JOIN (SELECT * FROM Enrolment WHERE lecturers = 2) AS TEMP2
      ON TEMP1.students = TEMP2.students
    ) AS TEMP
ON TEMP.lecturers = Lecturer_id
```

# Motivation

## Security policies

- Suppose a security policy such that the ids, names and emails of lecturers are public information, but the students taught by each lecturers are confidential information.

# Motivation

## Security policies

- Suppose a security policy such that the ids, names and emails of lecturers are public information, but the students taught by each lecturers are confidential information.
- Suppose that executing Query#1, Query#2, and Query#3, we obtain the same result, namely,

```
+-----+
| email  |
+-----+
| Juan@email.com |
+-----+
```

# Motivation

## Executing queries

In this case, for a person who knows the “meaning” of these queries:

- Query#1 reveals that the email *Juan@email.com* belongs to a lecturer named *Juan*.
- Query#2 reveals that the lecturer with email *Juan@email.com* (and whose *id* is 1) teaches the student whose *id* is 1.
- Query#3 reveals that there is at least one student who is taught both by the lecturer with email *Juan@email.com* (and whose *id* is 1) and the lecturer whose *id* is 2.

# Motivation

## “Unsecured” queries

In this case, for a person who knows the “meaning” of these queries:

- Query#1 reveals that the email *Juan@email.com* belongs to a lecturer named *Juan*. [compliant]
- Query#2 reveals that the lecturer with email *Juan@email.com* (and whose *id* is 1) teaches the student whose *id* is 1. [non-compliant]
- Query#3 reveals that there is at least one student who is taught both by the lecturer with email *Juan@email.com* (and whose *id* is 1) and the lecturer whose *id* is 2. [non-compliant]

# Remarks

Enforcing *fine-grained access control* (FGAC) policies on SQL queries is not trivial.

- no standard for enforcing FGAC policies on SQL queries, currently: Role-based Access Control model <sup>2</sup>.

---

<sup>2</sup>INCITS 359-2012. Information Technology - Role Based Access Control — standardized by InterNational Committee for Information Technology

Enforcing *fine-grained access control* (FGAC) policies on SQL queries is not trivial.

- no standard for enforcing FGAC policies on SQL queries, currently: Role-based Access Control model <sup>2</sup>.
- checking that the final result for policy-compliant is not enough! (the fact that a result is obtained can reveal confidential information, e.g., the students taught by a lecturer).

---

<sup>2</sup>INCITS 359-2012. Information Technology - Role Based Access Control — standardized by InterNational Committee for Information Technology

# Our model-driven approach

A model-based characterization for enforcing FGAC on SQL queries<sup>3</sup>

We define a predicate  $\text{AuthQuery}(\mathcal{S}, u, r, q, db)$  that represents whether

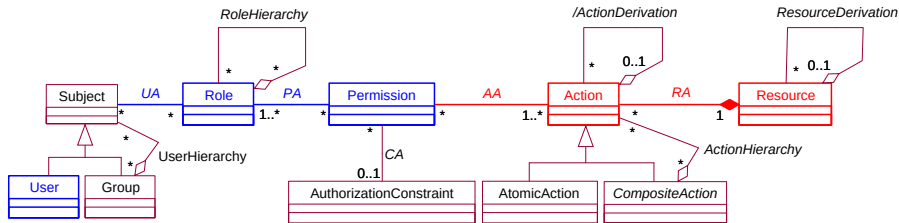
- a user  $u$
- with the role  $r$
- is authorized to execute a SQL query  $q$
- over the database state  $db$
- with respect to the FGAC policy model  $\mathcal{S}$ .

---

<sup>3</sup>Nguyen et. al. Model-based characterization of fine-grained access control authorization for SQL queries. JOT 2021.

# Our model-driven approach

SecureUML <sup>4</sup>

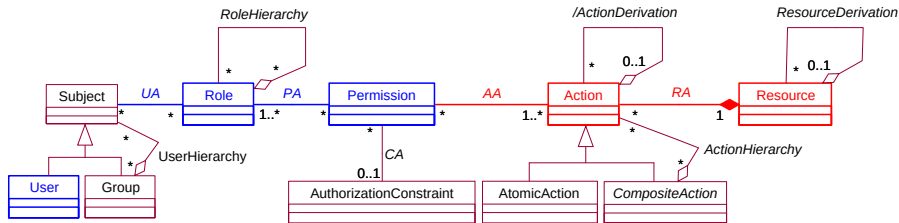


In our approach, we use *SecureUML* for modelling FGAC policies.

<sup>4</sup>Lodderstedt et. al. , SecureUML: A UML-Based Modelling Language for Model-Driven Security. UML 2002

# Our model-driven approach

SecureUML <sup>4</sup>



In our approach, we use *SecureUML* for modelling FGAC policies.

- key to our approach is that *authorization constraints* (i.e, the conditions that need to be satisfied by the current state of the database) are expressed in OCL.

<sup>4</sup>Lodderstedt et. al. , SecureUML: A UML-Based Modelling Language for Model-Driven Security. UML 2002

# Our model-driven approach

Example: Query#1

```
SELECT email FROM Lecturer WHERE name = 'Juan'
```

- For every lecturer instance in  $db$ , user  $u$  with the role  $r$  is authorized to read the **name** attribute, according to the security model  $\mathcal{S}$ .

# Our model-driven approach

## Example: Query#1

```
SELECT email FROM Lecturer WHERE name = 'Juan'
```

- For every lecturer instance in  $db$ , user  $u$  with the role  $r$  is authorized to read the name attribute, according to the security model  $\mathcal{S}$ .
- For every lecturer instance in  $db$  whose name is 'Juan', user  $u$  with the role  $r$  is authorized to read the email attribute, according to the security model  $\mathcal{S}$ .

# The function SecQuery()

## The idea

We define a function  $\text{SecQuery}(\mathcal{S}, q)$  that implements the characterization  $\text{AuthQuery}(\mathcal{S}, u, r, q, db)$ . In a nutshell,  $\text{SecQuery}(\mathcal{S}, q)$  transform  $q$  into a secured stored-procedure  $\lceil \text{SecQuery}(\mathcal{S}, q) \rceil()$  such that:

# The function SecQuery()

## The idea

We define a function  $\text{SecQuery}(\mathcal{S}, q)$  that implements the characterization  $\text{AuthQuery}(\mathcal{S}, u, r, q, db)$ . In a nutshell,  $\text{SecQuery}(\mathcal{S}, q)$  transform  $q$  into a secured stored-procedure  $\lceil \text{SecQuery}(\mathcal{S}, q) \rceil()$  such that:

- if the user  $u$  with the role  $r$  is authorized according to  $\mathcal{S}$  to execute the query  $q$  on the database  $db$ , i.e.,  $\text{AuthQuery}(\mathcal{S}, u, r, q, db)$  holds, then

$$\text{Exec}(db, \lceil \text{SecQuery}(\mathcal{S}, q) \rceil(u, r)) = \text{Exec}(db, q)$$

# The function SecQuery()

## The idea

We define a function  $\text{SecQuery}(\mathcal{S}, q)$  that implements the characterization  $\text{AuthQuery}(\mathcal{S}, u, r, q, db)$ . In a nutshell,  $\text{SecQuery}(\mathcal{S}, q)$  transform  $q$  into a secured stored-procedure  $\lceil \text{SecQuery}(\mathcal{S}, q) \rceil()$  such that:

- if the user  $u$  with the role  $r$  is authorized according to  $\mathcal{S}$  to execute the query  $q$  on the database  $db$ , i.e.,  $\text{AuthQuery}(\mathcal{S}, u, r, q, db)$  holds, then

$$\text{Exec}(db, \lceil \text{SecQuery}(\mathcal{S}, q) \rceil(u, r)) = \text{Exec}(db, q)$$

- otherwise,

$\text{Exec}(db, \lceil \text{SecQuery}(\mathcal{S}, q) \rceil(u, r))$  returns an error

# The function SecQuery()

## The implementation

- OCL authorization constraints are mapped into SQL <sup>5</sup>.

---

<sup>5</sup>Nguyen et. al. OCL2PSQL: An OCL-to-SQL Code-Generator for Model-Driven Engineering. FDSE 2019.

# The function SecQuery()

## The implementation

- OCL authorization constraints are mapped into SQL <sup>5</sup>.
- SQL generated authorization checks are executed within the stored-procedure when accessing data.
  - ▶ if a check fails, then an error is signalled.

---

<sup>5</sup>Nguyen et. al. OCL2PSQL: An OCL-to-SQL Code-Generator for Model-Driven Engineering. FDSE 2019.

# The function SecQuery()

## The implementation

- OCL authorization constraints are mapped into SQL <sup>5</sup>.
- SQL generated authorization checks are executed within the stored-procedure when accessing data.
  - ▶ if a check fails, then an error is signalled.
- Our approach follows *Non-Truman* model: either you are authorized to see everything or you see nothing!
  - ▶ this is in contrast to *Truman* model, e.g., materialized views and query filtering.

---

<sup>5</sup>Nguyen et. al. OCL2PSQL: An OCL-to-SQL Code-Generator for Model-Driven Engineering. FDSE 2019.

# The function SecQuery()

Query#1: SELECT email FROM Lecturer WHERE name = 'Juan'

```
CREATE PROCEDURE 「SecQuery( $\mathcal{S}$ , Query#1)」 (in user, in role)
:
CREATE TEMPORARY TABLE TEMP1 AS (
  SELECT * FROM Lecturer
  WHERE (CASE 「AuthFunc( $\mathcal{S}$ , name)」(user, role, Lecturer_id)
    WHEN TRUE THEN name ELSE throw_error() END) = 'Juan'
);

CREATE TEMPORARY TABLE TEMP2 AS (
  SELECT CASE 「AuthFunc( $\mathcal{S}$ , email)」(user, role, Lecturer_id)
    WHEN TRUE THEN email ELSE throw_error() END AS email
  FROM TEMP1
);
:
END
```

# The function AuthFunc()

```
CREATE FUNCTION  $\lceil$ AuthFunc( $S$ ,email) $\rceil$  (user, role, self)
RETURNS INT DETERMINISTIC
  IF (role =  $r_1$ )
    THEN IF ( $\lceil$ AuthFuncRole( $S$ ,email, $r_1$ ) $\rceil$ (self, user))
      THEN RETURN (1);
      ELSE RETURN (0);
    END IF;
  ELSE IF (role =  $r_2$ )
    :
  ELSE RETURN 0;
  END IF;
END
```

# The function AuthFuncRole()

```
CREATE FUNCTION  $\lceil$ AuthFuncRole( $\mathcal{S}$ , email,  $r_1$ ) $\rceil$  (self, user)
RETURNS INT DETERMINISTIC
BEGIN
  DECLARE result INT DEFAULT 0;
  SELECT res INTO result
    FROM ( map(auth) ) AS TEMP;
  RETURN result;
END
```

where

- *auth* is the OCL authorization constraint in  $\mathcal{S}$  for a user with role  $r_1$  to read a lecturer's email, and
- $\text{map}(\text{auth})$  is the mapping of *auth* into SQL.

# An extended model-based characterization of fine-grained access control for SQL queries

We extend our model-driven approach to support *association-classes*:

- 1 the underlying data model to support *association-classes* definition.
- 2 the FGAC policies' security modeling language to support actions on protected resources for *association-classes*.
- 3 the characterization of FGAC for SQL queries that includes *association-classes* elements.

# Conclusions

- We have proposed a formal definition/*characterization* of FGAC-*authorization* for SQL queries.
  - ▶ for a subset of the SQL-SELECT statements.

# Conclusions

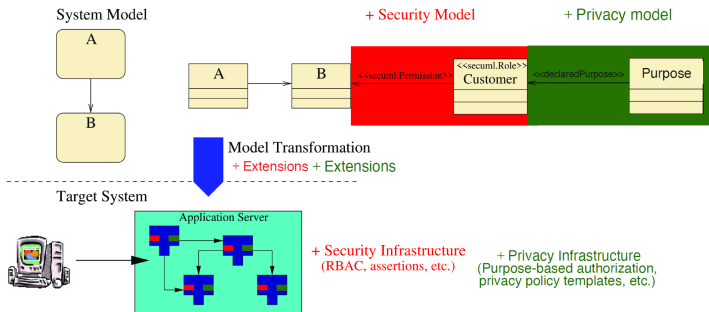
- We have proposed a formal definition/*characterization* of FGAC-*authorization* for SQL queries.
  - ▶ for a subset of the SQL-SELECT statements.
- We have proposed a model-driven approach for *enforcing* FGAC-policies when executing SQL queries.
  - ▶ modelling policies using SecureUML and OCL,
  - ▶ generating “secure” queries using code-generators.

# Conclusions

- We have proposed a formal definition/*characterization* of FGAC-*authorization* for SQL queries.
  - ▶ for a subset of the SQL-SELECT statements.
- We have proposed a model-driven approach for *enforcing* FGAC-policies when executing SQL queries.
  - ▶ modelling policies using SecureUML and OCL,
  - ▶ generating “secure” queries using code-generators.
- We have extended both our characterization and implementation to support *association-classes*.

# On-going work (selected)

- Extend our approach with other SQL statements and languages <sup>6</sup>.
- Incorporate privacy concepts into our model-driven approach <sup>7</sup>.



<sup>6</sup>Piskor Karlo. Secured Object Relational Mappers. Bachelor Thesis. ETH 2023

<sup>7</sup>Srdan Krstic, Hoang Nguyen and David Basin. Model-driven Privacy. PETS 2024(1).

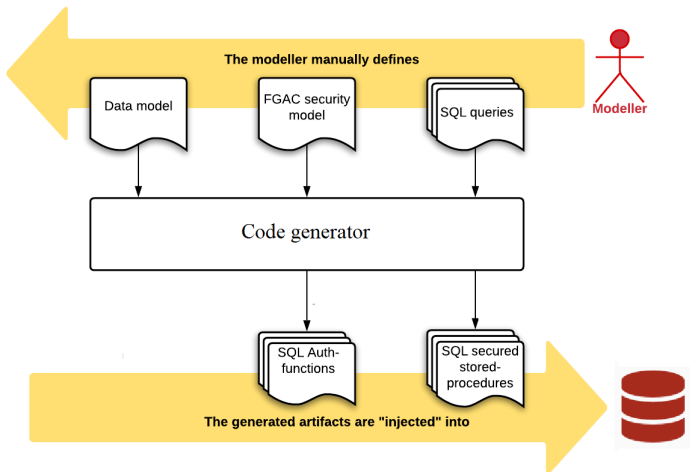
# The correctness of mapping from OCL to SQL

Given a contextual model  $\mathcal{D}$ , let  $\mathcal{O}$  be an object model and  $exp$  be an OCL expression of  $\mathcal{D}$ . Let  $map(\mathcal{O})$  be an implementation of  $\mathcal{O}$  in SQL. Then, we say that  $map(exp)$  is correctly implementing  $exp$  if and only if:

$$\text{Eval}(exp, \mathcal{O}) =_{\text{map}()} \text{Exec}(map(exp), map(\mathcal{O}))$$

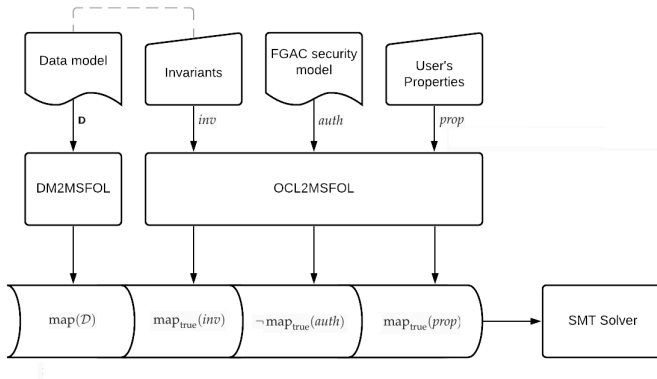
- Proof completed in pen and paper fashion.
- Formalization in Isabelle/HOL: in progress.

# SQLSI: In a nutshell



# Optimizing FGAC enforcement: In a nutshell

We identify scenarios in which security access control policies can be “skipped”. These scenarios can be found via model-checking.



# Proving the correctness of SQL implementations of OCL constraints

Main idea: to move everything to the FOL world and use solvers to prove for implementation correctness.

